Mastering Layouts

Based on Chapters 28-33

Muhammad Naveed

Agenda

- → Composing with **Row** & **Column**
- → Mastering the **Box** Layout
- → Building Responsive UIs with **Flow Layouts**
- → Creating **Custom Layout Modifiers**
- → Building **Custom Layout Composables**

The Fundamentals

Row (Horizontal)

The Row composable lays out its children horizontally in a sequence.

Key Parameters:

- horizontalArrangement: Positions children along the x-axis.
- verticalAlignment: Aligns children along the cross-axis (y-axis).

Column (Vertical)

The Column composable stacks its children vertically.

Key Parameters:

- verticalArrangement: Positions children along the y-axis.
- horizontalAlignment: Aligns children along the cross-axis (x-axis).

Alignment & Arrangement

Alignment

Controls positioning on the **cross-axis**. Options include Start, Center, and End. Think of it as gravity acting on the items.

Arrangement

Controls positioning on the **main-axis**. Includes spacing options like SpaceBetween, SpaceAround, and SpaceEvenly.

Weights

The Modifier.weight() scope modifier distributes available space proportionally among siblings, allowing flexible UIs.

The Box Layout

Stacking & Depth

Unlike Row and Column, the Box composable stacks its children on top of each other. The order of declaration determines the Z-order, with the last declared item rendering on top.

Precise Positioning

The contentAlignment parameter allows for 9 distinct positions (e.g., TopStart, Center, BottomEnd), making it ideal for overlays, badges, and background images.

Visual Styling: Shapes & Clips

CircleShape

`Modifier.clip(CircleShape)` to create perfect avatars or circular buttons.

RoundedCorner

`RoundedCornerShape(x.dp)` softens UI edges for a modern, friendly aesthetic.

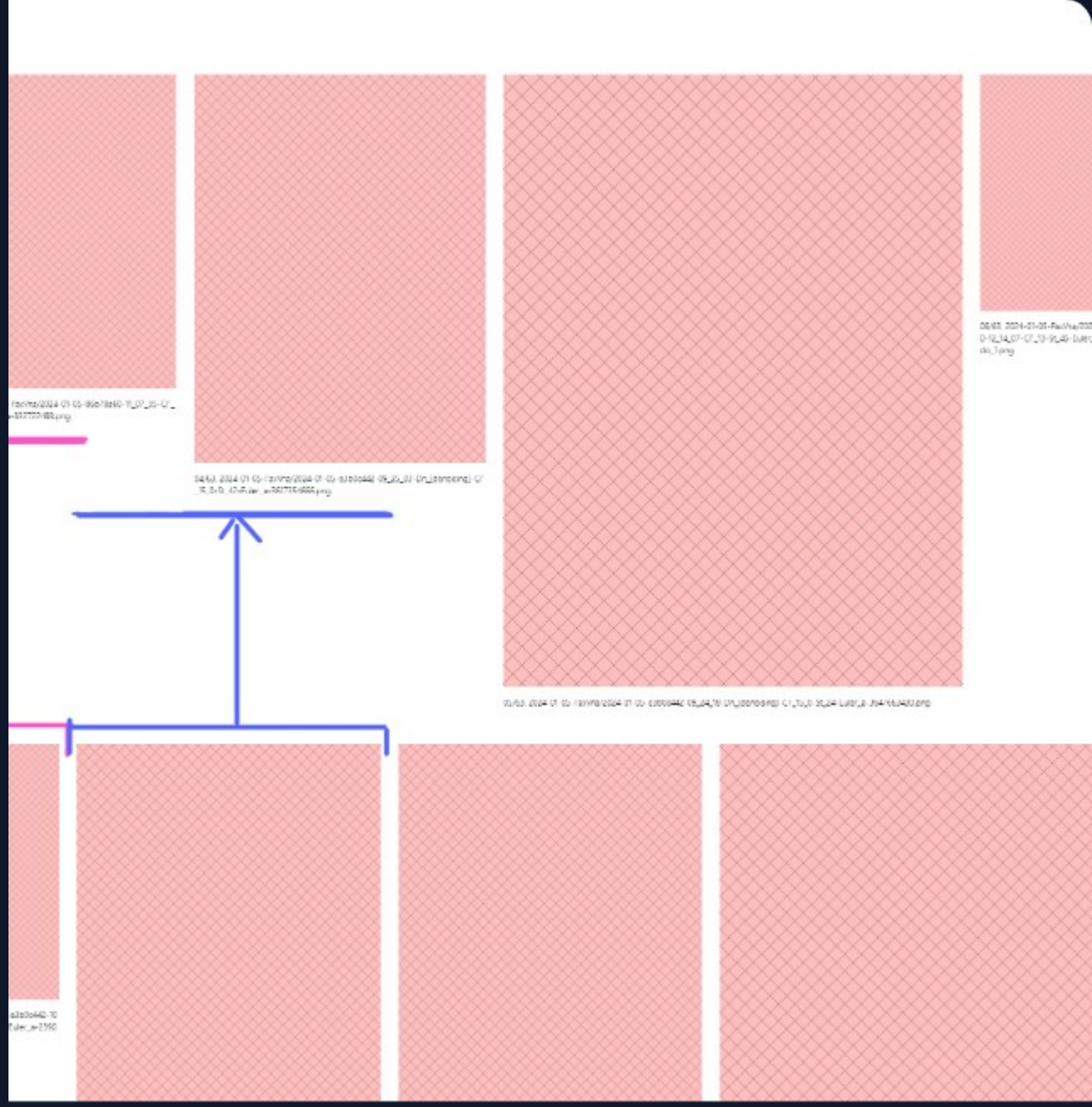
CutCorner

`CutCornerShape(x.dp)` creates angular, aggressive edges for distinct branding.

Flow Layouts

Standard Rows and Columns are rigid; they clip content when they run out of space. FlowRow and FlowColumn solve this by automatically wrapping content to the next line or column.

This behavior is essential for displaying dynamic collections like tags, keywords, or responsive grids that must adapt to different screen sizes without horizontal scrolling.



Flow Layout Capabilities

- **Dual Axis Control:** Configure both main-axis and cross-axis arrangement to control spacing and alignment in all directions.
- **Max Items Constraints:** Use `maxItemsInEachRow` to strictly limit the number of items before forcing a wrap.
- **Fractional Sizing:** Modifiers like `fillMaxWidth(0.5f)` allow items to occupy specific percentages of the flow track.
- **Dynamic Weights:** Just like in standard layouts, weights can be assigned to flow items to expand and fill empty space.

Beyond Standard Composables

When Row, Column, and Box aren't enough...

Custom Layout Modifiers

The Concept

Layout modifiers allow you to intercept the measurement and placement phase of a single child element. You can calculate new X, Y coordinates relative to the parent's default placement logic, effectively creating custom positioning behavior.

The Syntax

It involves the layout lambda which provides access to the measurable and constraints.

```
fun Modifier.custom() = layout {
    measurable, constraints ->
    val placeable = measurable.measure(constraints)
    layout(width, height {
        placeable.placeRelative(x, y)
    }
}
```

Building Custom Layouts

The Layout Composable

To position *multiple* children with custom logic, use the Layout composable. It allows you to measure all children (converting measurables to placeables) and then define exactly where each one sits on the canvas.

Example: CascadeLayout
An example explored in Chapter 33 involves placing each child at an indented offset relative to the previous one, creating a diagonal "staircase" or cascading effect.

Questions!

Thanks for your attention